

Речь на преддипломной защите

QuantumTrader Pro • ~10 минут • Редактируй под себя

Слайд 1 — Титульный (30 сек)

Добрый день. Меня зовут Евгений, тема моей дипломной работы — **QuantumTrader Pro**, торговый терминал реального времени, разработанный на языке C++ с использованием фреймворка Qt и аппаратного рендеринга через OpenGL.

Терминал подключается к криптобирже Bybit по WebSocket и отображает рыночные данные в реальном времени.

Слайд 2 — Актуальность (1 мин)

Почему это актуально? Существующие браузерные торговые терминалы имеют три принципиальных ограничения.

Во-первых, скорость обработки данных. Биржа Bybit отправляет обновления тысячи раз в секунду. Браузерные решения на JavaScript не успевают обрабатывать такой поток без потерь.

Во-вторых, рендеринг графики. Когда на экране одновременно работают график свечей, стакан заявок и лента сделок — нужна аппаратная графика. CSS и Canvas с этим не справляются.

В-третьих, расширяемость. Профессиональные трейдеры хотят писать собственные индикаторы, не дожидаясь обновлений от разработчика.

Решение — нативный десктопный терминал на C++, где всё это решается на уровне архитектуры.

Слайд 3 — Архитектура (1.5 мин)

Система построена в три слоя.

Первый — источник данных. Bybit WebSocket API: kline (свечи), orderbook (стакан), publicTrade (лента сделок).

Второй — бизнес-логика. BybitConnector управляет соединением. ByBitParser разбирает JSON через simdjson. MarketDataManager управляет подписками. EventBus — шина событий на Qt-сигналах, связывающая компоненты без прямых зависимостей.

Третий — интерфейс. ChartContainer, OrderBookContainer, TimeAndSalesContainer. WindowManager на Qt ADS управляет расположением окон.

Главное преимущество: модули не знают друг о друге. Добавить новый виджет или биржу можно, не трогая существующий код.

Слайд 4 — Технологический стек (1 мин)

C++17 — производительность нативного кода, RAII, zero-cost абстракции.

Qt 6 — сигналы/слоты для событийной архитектуры, Qt ADS для dock-панелей.

OpenGL — GPU-рендеринг через QOpenGLWidget. Тысячи свечей без падения FPS.

simdjson — SIMD-инструкции CPU. Биржевые сообщения парсятся за микросекунды.

WebSocket — асинхронный push-канал от Bybit в реальном времени.

CMake с GLOB_RECURSE — новые файлы подхватываются при сборке автоматически.

Слайд 5 — Реализованные модули (2 мин)

График свечей. QOpenGLWidget — GPU рисует каждую свечу напрямую. История через REST + live через WebSocket. Масштабирование колесиком, все таймфреймы Bybit.

Стакан заявок (Order Book). Полный снимок + инкрементальные delta-обновления. Три независимо перетаскиваемых колонки: Price, Size, Total. Индикатор объема в виде цветного бара. При любом сужении окна все три колонки остаются видимы.

Лента сделок (Time and Sales). Данные через publicTrade Bybit. Зеленый фон — покупка, красный — продажа. Авто-скролл, ручная прокрутка, буфер 500 тиков.

Слайд 6 — Производительность (1 мин)

Три ключевых показателя.

Менее 1 микросекунды — парсинг одного JSON-сообщения через simdjson SIMD. Примерно в 10 раз быстрее стандартных парсеров.

60 FPS — рендеринг интерфейса на GPU. Основной поток не загружается.

Менее 50 миллисекунд — задержка данных от биржи через WebSocket.

Архитектурно: EventBus устраняет лишние зависимости, абсолютные пиксели колонок исключают пересчет на каждый фрейм, паттерн Container/Widget разделяет подписку и рендеринг.

Слайд 7 — Перспективы (1 мин)

Система спроектирована с расчетом на развитие.

Lua-скриптинг индикаторов. Трейдер пишет индикатор на Lua без знания C++. C++-хост предоставляет API: данные свечей, функции EMA и SMA, рисование линий на графике. Скрипт загружается в рантайме без перекомпиляции. Архитектура EventBus уже готова к этому.

Дополнительно: алерты по цене, поддержка нескольких бирж через IConnector, Watchlist с live-ценами.

Слайд 8 — Заключение (30 сек)

В ходе работы реализован полноценный торговый терминал с тремя модулями отображения рыночных данных. Применены современные технологии: C++17, OpenGL, simdjson, Qt ADS. Архитектура на EventBus обеспечивает независимость компонентов и легкую расширяемость.

Спасибо за внимание. Готов ответить на вопросы.

Возможные вопросы и ответы

— Почему C++, а не Python или Java?

C++ даёт прямой контроль над памятью и производительность без garbage collector. Для торгового терминала, где каждая миллисекунда важна, это принципиально.

— Как обеспечивается потокобезопасность?

WebSocket-соединения работают в отдельных потоках пула. EventBus использует Qt-сигналы с Qt::QueuedConnection — это автоматически маршрутизирует вызовы в нужный поток.

— Почему именно Bybit?

Bybit имеет подробную документацию WebSocket API и работает без аутентификации для публичных данных. Архитектура через IConnector позволит добавить другие биржи в будущем.

— Что такое delta-обновления в стакане?

Биржа сначала присылает полный снимок стакана (snapshot), а затем только изменения (delta) — уровни, которые появились или исчезли. Это экономит трафик и ускоряет обновление.

— Тестировалась ли система под нагрузкой?

Система тестировалась на реальных данных Bybit в реальном времени. При одновременной работе графика, стакана и ленты сделок по паре BTCUSDT рендеринг стабилен на 60 FPS.